

Additional Technical Appendices for EXGUARD

Anonymous Author(s)

Affiliation

Address

email

Q Tradeoffs Between Smoothness and Predictor Usage

Table 1: Comparison of Smoothness and Predictor Usage.

NRT	Smoothness	#Pred.
P.M.	$\mathcal{O}(\sqrt{\varphi_t})$	$\mathcal{O}(H_k \cdot \text{OPT})$
LM.	$\mathcal{O}(\log(\varphi_t))$	$\mathcal{O}(\text{OPT})$
LNonM.	$\mathcal{O}(\log(k)/k \cdot \varphi_t)$	<i>unbounded</i>
B.O.	$\mathcal{O}(1/k \cdot \varphi_t)$	<i>unbounded</i>
B.O.&LRU	$\mathcal{O}(1/k \cdot \varphi_t)$	$\mathcal{O}(k \cdot \text{OPT})$
B.O.&M. ^D	$\mathcal{O}(1/k \cdot \varphi_t)$	$\mathcal{O}(H_k \cdot \text{OPT})$
B.O.&M. ^R	$\mathcal{O}(1/k \cdot \varphi_t)$	$\mathcal{O}(H_k \cdot \text{OPT})$
B.O.&EQ.	$\mathcal{O}(1/k \cdot \varphi_t)$	$\mathcal{O}(H_k \cdot \text{OPT})$
G.&B.O.	$\mathcal{O}(\log(\varphi_t))$	$\mathcal{O}(\text{OPT})$
EXG.&B.O.	$\mathcal{O}(\min(\log(\varphi_t), \lambda\sqrt{\varphi_t}))$	$\mathcal{O}(d \cdot \text{OPT})$
Binary	Smoothness	#Pred.
M.P.	$\mathcal{O}(H_k \cdot \varphi_b)$	$\mathcal{O}(H_k \cdot \text{OPT})$
MARK0	$\mathcal{O}(H_k \cdot \varphi_b)$	<i>unbounded</i>
G.&LRB	$\mathcal{O}(H_k \cdot \varphi_b)$	$\mathcal{O}(\text{OPT})$
EXG.&LRB	$\mathcal{O}(H_k/d \cdot \varphi_b)$	$\mathcal{O}(d \cdot \text{OPT})$
Action	Smoothness	#Pred.
F&R ($f(i) = 2^i - 1$)	$\mathcal{O}(\log(\varphi_a))$	$\mathcal{O}(\sqrt{k} \cdot \text{OPT})$
F&R	$\mathcal{O}(f^{-1}(\varphi_a))$	$\mathcal{O}(f(\log k) \cdot \text{OPT})$
FitF	Smoothness	#Pred.
F&R (FitF)	$\mathcal{O}(\log(k)/b \cdot \varphi_f)$	$\mathcal{O}(b \cdot \text{OPT})$
G.&PA.	$\mathcal{O}(H_k \cdot \varphi_t)$	$\mathcal{O}(\text{OPT})$
EXG.&PA.	$\mathcal{O}(H_k/d \cdot \varphi_f)$	$\mathcal{O}(d \cdot \text{OPT})$

Notations in Table 1. Abbreviations include BLINDORACLE (B.O.), PREDICTIVEMARKER (P.M.), LNONMARKER (LNonM.), LMARKER (LM.), BLINDORACLE&MARKER (B.O.&M.), EQUI-TABLE (EQ.), MARK&PREDICT (M.&P.), PARROT (PA.), GUARD (G.), and EXGUARD (EXG.).

#Pred. represents the upper bound of the number of predictor calls. F&R’s smoothness depends on $f(i)$, which limits predictor calls. Our experiments focus on the F&R variant with optimal smoothness ($f(i) = 2^i - 1$). For clarity, we denote η_t/OPT as φ_t , η_b/OPT as φ_b , η_c/OPT as φ_a , and η_f/OPT as φ_f , where $\eta_t, \eta_b, \eta_c, \eta_f$ represent different types of prediction errors.

For EXGUARD, parameter $d \in [1, H_k]$. For F&R (FitF), parameter $b \in \{1, \dots, \log k\}$. The parameter λ in EXG.&B.O.’s smoothness can be refined to $\lambda = h/e^h \leq 1/e$ (see Theorem S.1), where $h = H_k/(2d)$.

Analysis of Predictor Usage. B.O. and MARK0. suffer from unbounded robustness and invoke the predictor on each cache miss, resulting in unbounded predictor usage. The predictor usage of F&R and F&R (FITF) has been analyzed in the corresponding paper.

P.M. invokes the predictor $\mathcal{O}(H_k \cdot \text{OPT})$ -times, as it evicts pages based on predictions whenever the corresponding eviction chain has length at most H_k . In contrast, LM. relies on predictions only once at the beginning of each eviction chain, resulting in $\mathcal{O}(\text{OPT})$ predictor queries. The predictor usage of LNONM. is unbounded, as it invokes the predictor once per eviction chain, and the number of eviction chains depends on the total prediction error.

M.&P. invokes the predictor on each cache miss, so its predictor usage bound is the product of $\mathcal{O}(\text{OPT})$ and its robustness. Similarly, the predictor usage of B.O.&A is bounded by the product of $\mathcal{O}(\text{OPT})$ competitive ratio of the classical algorithm A, where $A \in \{\text{LRU}, \text{MARKER}, \text{EQ.}\}$.

Favorable Tradeoffs Achieved by EXGUARD. For algorithms relying on NRT predictions, EXG.&B.O. achieves better tradeoffs than P.M. and LM. as their tradeoffs are theoretically directly comparable. To compare with other algorithms such as B.O.&LRU, we conduct experiments, and the results indicate that EXG.&B.O. achieves the best empirical tradeoff.

Moreover, both EXG.&B.O. and G.&B.O. exhibit good smoothness, even with $\mathcal{O}(\text{OPT})$ predictor usage. This enhances practicality, especially when predictor calls are costly.

Among algorithms that rely on binary predictions or the FitF predictor, both EXG.&LRB and EXG.&PA. achieve state-of-the-art tradeoffs.

R Experimental Results for EXGUARD

We empirically compare EXGUARD&B.O. with other algorithms, with d set to H_k .

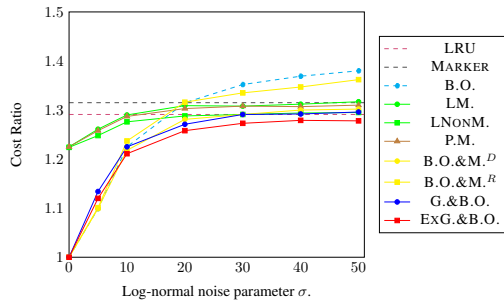


Figure 1: Performance with synthetic predictions of NRT on BrightKite.

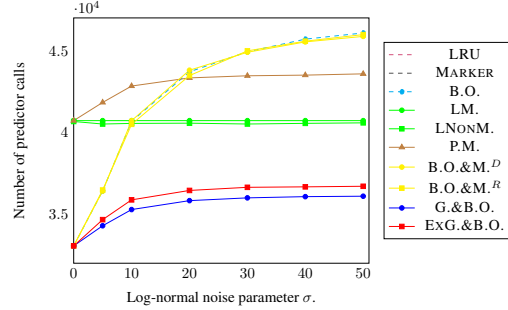


Figure 2: Number of predictor calls with synthetic predictions of NRT on BrightKite.

Table 2: Average cost ratios ($\overline{\text{CR}}$), average LRU-normalized cost ratios ($\overline{\text{LCR}}$), and average number of predictor calls ($\overline{\text{\#Pred.}}$) on SPEC CPU2006 Benchmark using PLECO predictor.

Metrics	LRU	B.O.	P.M.	LM.	LNONM.	B.O.&M. ^D	G.&B.O.	EXG.&B.O.
$\overline{\text{CR}}$	1.478	1.404	1.335	1.335	1.346	1.294	1.226	1.225
$\overline{\text{LCR}}$	1	1.049	0.946	0.942	0.951	0.766	0.627	0.623
$\overline{\text{\#Pred.}}$	0	79488	75870	72682	72767	79488	62099	62553

Improved Performance. Figure 1, 2 and Table 2 show that EXGUARD&B.O. outperforms GUARD&B.O. through increased predictor usage, empirically validating its improved smoothness. It is noteworthy that G.&B.O. and EXG.&B.O. depend less on the predictor than other algorithms utilizing next request time (NRT) predictions.

In the above, B.O.&M.^R is excluded due to underperforming relative to B.O.&M.^D. F&R is excluded due to its reliance on action prediction. Although action predictions can be generated using (NRT), doing so requires NRT predictions for all requested pages. In contrast, other algorithms only

40 predict the NRT of cached pages at eviction (predictions can be reused if not requested). Thus, their
 41 predictor call counts are not comparable.

42 **S Tighter Smoothness of EXGUARD&BLINDORACLE**

43 The following theorem presents a tighter smoothness result than the one in Appendix L.4.

44 **Theorem S.1.** EXGUARD&BLINDORACLE is $\mathcal{O}(\min(\log(\eta_t/\text{OPT}), h/e^h \cdot \sqrt{\eta_t/\text{OPT}}))$ -smooth,
 45 where $h = H_k/(2d)$.

46 *Proof.* For an old-page eviction chain $\mathcal{E}_q$ in phase q with x evictions based on predictions, the
 47 expected length of it is $|\mathcal{E}_q| \leq x \cdot H_k/d$. When there are z unrequested old pages in the cache,
 48 the expected number of subsequent random evictions satisfies $L(z) \leq H_z < \log_2(z)$. (Refer to
 49 Appendix J for $L(z)$.) Thus, if a prediction-based eviction leads to y subsequent random evictions, it
 50 contributes at least an expected e^y inversions.

51 From the above, if $\mathcal{E}_q$ contributes $I(\mathcal{E}_q)$ inversions, then $I(\mathcal{E}) \geq \mathcal{O}(x^2 \cdot e^{H_k/d})$. Therefore, $|\mathcal{E}_q| \leq$
 52 $x \cdot H_k/d \leq \mathcal{O}\left(\frac{H_k}{d} \cdot \sqrt{\frac{I(\mathcal{E}_q)}{e^{H_k/d}}}\right)$.

53 The total number of inversions satisfies $\text{inv}(\theta, \hat{\theta}) \geq \sum_{q \in \mathcal{Q}} \sum_{\mathcal{E}_q} I(\mathcal{E}_q)$. Let N denote the total number of
 54 old-page eviction chains, where $\text{OPT} \leq N \leq 2\text{OPT}$, as shown in Equation (12) of Appendix K.2.

55 We then bound the total cost of EXGUARD&BLINDORACLE, denoted as $\mathbb{E}[\text{EXG\&B}]$, by

$$\begin{aligned} \mathbb{E}[\text{EXG\&B}] &\leq \sum_{q \in \mathcal{Q}} c_q + \sum_{q \in \mathcal{Q}} \sum_{\mathcal{E}_q} \mathcal{O}(H_k/d \cdot \sqrt{I(\mathcal{E}_q)/e^{H_k/d}}) \quad (\text{by Lemma J.1}) \\ &\leq \sum_{q \in \mathcal{Q}} c_q + N \cdot \mathcal{O}\left(H_k/(de^{H_k/(2d)}) \cdot \sqrt{\frac{\text{inv}(\theta, \hat{\theta})}{N}}\right) \quad (\text{by Jensen's inequality}) \\ &= \mathcal{O}\left(\text{OPT} \cdot h/e^h \cdot \sqrt{\frac{\eta_t}{\text{OPT}}}\right), \quad (\text{by Lemma K.3}) \end{aligned} \tag{1}$$

56 where $h = H_k/(2d)$. EXG.&B.O. inherits the $\mathcal{O}(\log(\eta_t/\text{OPT}))$ smoothness of GUARD&B.O.,
 57 adding at most $\mathcal{O}(d)$ evictions per old-page chain, where $1 \leq d \leq H_k$. Combining the two bounds
 58 above completes the proof.

59 □